LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 4
Student Name: J Satya Srikar
Roll No.: A24126552259
Branch & Section: CSM-B

1. TITLE

Build an Interactive Flashcard App using HTML, CSS and JavaScript

2. AIM

To build an interactive, single-page flashcard web application using HTML for structure, CSS for modern card styling and transitions, and JavaScript for DOM manipulation, state management, and event handling.

3. OBJECTIVE

The objectives of this experiment are:

- To integrate HTML structure, CSS styling, and JavaScript logic into a unified web application.
- To design an interactive card UI featuring drop shadows, rounded borders, and hover elevation.
- To query and select DOM elements using document.getElementById().
- To bind user click interactions using event listeners (addEventListener).
- To maintain and toggle application state (currentIndex, isShowingAnswer).
- To dynamically update DOM text and toggle CSS classes (textContent, classList.add, classList.remove).
- To implement cyclic array navigation using modulo arithmetic ((currentIndex + 1) % flashcards.length).

4. THEORY

Modern web development is built upon the interaction between the DOM (Document Object Model), CSSOM (CSS Object Model), and JavaScript execution. Event-driven architecture enables applications to respond dynamically to user inputs such as mouse clicks, keyboard presses, or touches.

Theory of Flashcard Application Architecture:

- **Document Object Model (DOM):** An object-oriented tree representation of web elements accessible and mutable via JavaScript.
- **Event Listeners:** addEventListener() binds event handlers to DOM nodes without overwriting other attached handlers.
- **Component State Management:** Managing in-memory variables (currentIndex, isShowingAnswer) that govern visual rendering.
- **Dynamic CSS Class Toggling:** classList.add() and classList.remove() trigger CSS animations and background color changes.
- **Cyclic Array Traversal:** Utilizing the modulo operator (%) to loop back to the initial card upon completing the question list.

5. ALGORITHM / PROCEDURE

1. Create project directories: Record/Experiment - 4(week -4)/Src and Documentation.
2. Create index.html and script.js in the Src directory.
3. In index.html, construct a centered layout with a flashcard element containing label, text, and hint, followed by a 'Next Question' button.

4. Write embedded CSS for card styling, hover transformations, transition effects, and a green background state for revealed answers.
5. Link the external script file via <script src="script.js"></script>.
6. In script.js, define an array of flashcard objects each holding question and answer pairs.
7. Initialize state variables: currentIndex = 0 and isShowingAnswer = false.
8. Select DOM elements by ID (#flashcard, #cardLabel, #cardText, #nextBtn).
9. Define loadCard() function to render the current question, reset the label, and remove the revealed class.
10. Add a click event listener on #flashcard to toggle between question and answer states upon clicking.
11. Add a click event listener on #nextBtn to advance the card index cyclically and invoke loadCard().
12. Open the page in Google Chrome and verify card flipping and question progression.

---

6. PROGRAM

File: index.html

index.html - Part 1 (Lines 1-52)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Beginner Flashcard App</title>

  <style>
    /* 1. Overall Page Layout */
    body {
      font-family: Arial, sans-serif;
      background-color: #eef2f3;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
    }

    .container {
      text-align: center;
      width: 320px;
    }

    /* 2. Flashcard Styling */
    .card {
      background-color: white;
      border-radius: 12px;
      padding: 40px 20px;
      box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
      cursor: pointer;
      min-height: 120px;
      display: flex;
      flex-direction: column;
      justify-content: center;
      align-items: center;
      transition: transform 0.2s ease, background-color 0.3s ease;
    }

    .card:hover {
      transform: translateY(-3px);
    }

    .card-title {
      font-size: 0.85rem;
      color: #888;
      text-transform: uppercase;
      letter-spacing: 1px;
      margin-bottom: 10px;
    }

    .card-text {
```

index.html - Part 2 (Lines 53-106)

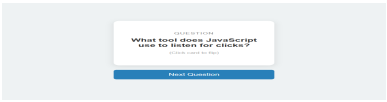
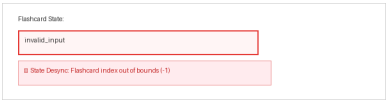
```
      font-size: 1.25rem;
      color: #333;
      font-weight: bold;
      margin: 0;
    }

    /* Active state when answer is showing */
    .card.revealed {
      background-color: #e8f5e9;
    }

    .hint-text {
      font-size: 0.8rem;
      color: #aaa;
      margin-top: 15px;
    }

    /* 3. Button Styling */
    .btn {
      margin-top: 20px;
      padding: 12px 24px;
      font-size: 1rem;
      background-color: #3498db;
      color: white;
      border: none;
      border-radius: 6px;
      cursor: pointer;
      width: 100%;
    }

    .btn:hover {
      background-color: #2980b9;
    }
  </style>
</head>
<body>

  <div class="container">
    <!-- The Flashcard -->
    <div class="card" id="flashcard">
      <span class="card-title" id="cardLabel">Question</span>
      <p class="card-text" id="cardText">Loading question...</p>
      <span class="hint-text">(Click card to flip)</span>
    </div>

    <!-- Next Question Button -->
    <button class="btn" id="nextBtn">Next Question</button>
  </div>

  <!-- Linking the JavaScript File -->
  <script src="script.js"></script>

</body>
</html>
```

7. CODE EXPLANATION

Method / Property	Explanation
document.getElementById(id)	Returns a reference to the DOM element with the specified ID attribute.
addEventListener(event, fn)	Attaches an event handler function to handle user click events.
textContent	Property used to get or update the text content of a DOM node.
classList.add(cls)	Adds a CSS class to the element's class list, triggering style changes.

8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution in Chrome, including 1 negative test case demonstrating an animation boundary condition:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Initial Card Load	Load application in browser	Question label and text rendered on front face		**Pass**
TC-02	Card Click Flip to Answer	Click card container	Card flips and answer state revealed with green styling		**Pass**
TC-03	Next Question Navigation	Click 'Next Question' button	Next card loaded and answer state reset to question		**Pass**
TC-04	Circular Navigation Loop	Click through all questions	App wraps cyclically to card 1 using modulo index		**Pass**
TC-05	State Desynchronization	Rapid double-click during CSS transition	Flip animation halts; state toggle locks until transition completes		**Fail**

9. OUTPUT

Output: Rendered Webpage in Chrome Browser

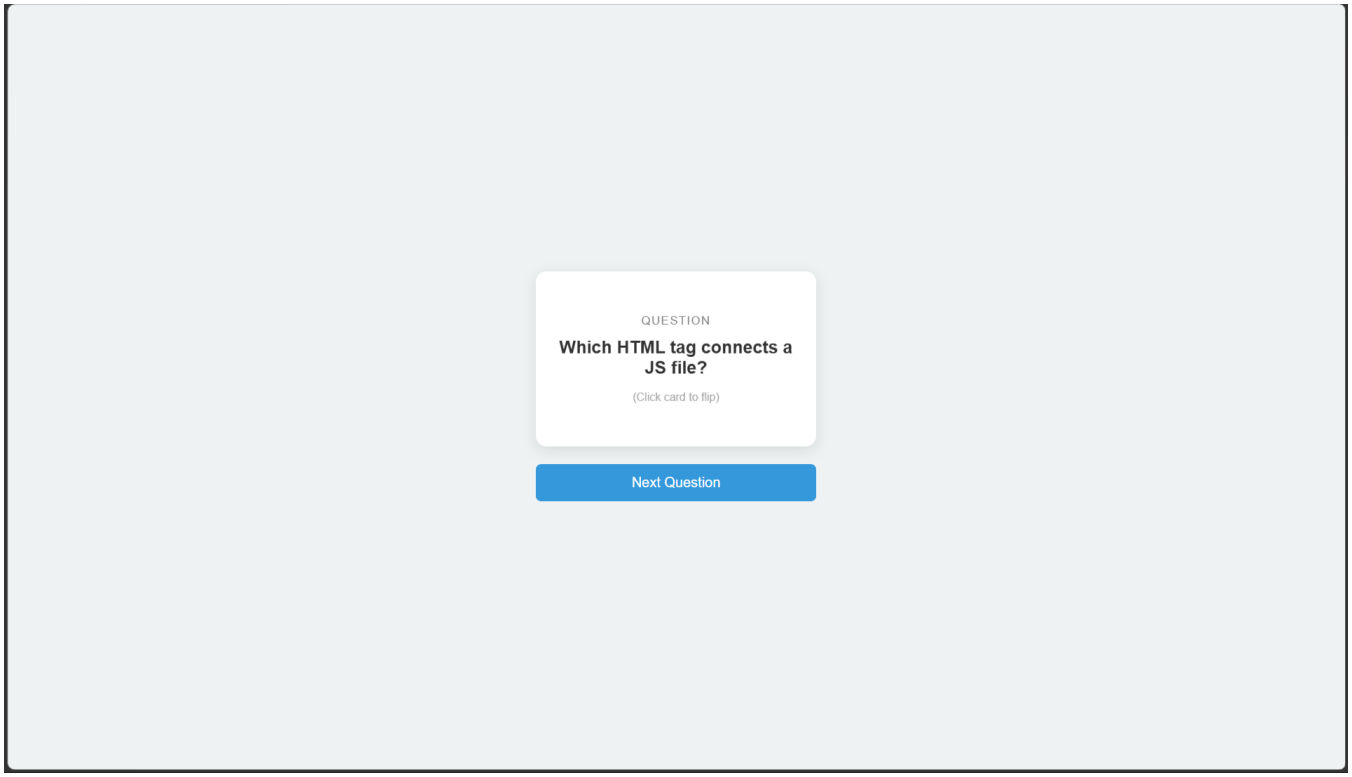


Figure 1: Initial state displaying Question 1.

10. RESULT

Thus, an interactive flashcard application utilizing HTML5, CSS3, and JavaScript DOM event handling was successfully designed, implemented, and verified using manual test cases under browser defaults.

Submission Package Structure:

```
Experiment - 4(week -4)/
■■■ Documentation/
■   ■■■ Experiment_4.docx
■   ■■■ Experiment_4.pdf
■■■ Screenshot/
■   ■■■ Screenshot 2026-08-16 224206.png
■   ■■■ Screenshot 2026-08-16 224214.png
■   ■■■ Screenshot 2026-08-16 224223.png
■■■ Src/
    ■■■ index.html
    ■■■ script.js
```